

Generative AI Analysis and Ethics Report

Generative approach: Transformer-based generation (character-level decoder)

Plain-English walkthrough (read this first if you're not a deep-learning engineer)

What this project does, in one sentence. It teaches a small AI to write text that *sounds like* the formal reports a security-operations analyst writes — but the AI's text is make-believe, not real information.

Who it's for. Imagine a security guard in a control room at 3 a.m., staring at alerts, who has to write a short formal paragraph explaining an incident so their manager will accept it. That paragraph has a recognizable *style* — dated, technical, calm, structured ("An incident was detected at... the team contained it by..."). This project does **not** write that paragraph for them. It only proves that a small AI can learn that *style* from public documents, so that a future, more careful system can be trusted to produce text in the right register.

How it learns — the autocorrect analogy. The AI is a "character-level Transformer." Think of your phone's autocorrect, which guesses the next word you're likely to type. This model does the same thing but guesses the next *letter* at a time, and it does it by paying attention to the last ~128 letters it has seen. "Transformer" is just the name of the guessing engine; nothing more mysterious than that. It learned by reading about 339,000 characters of real, public security-incident-handling documents (mostly NIST's official incident-response guides) over and over, 30 times, until it could reliably predict the next letter.

What the training material was. Public documents anyone can download: the U.S. government's official incident-handling guides (NIST SP 800-61), a European cybersecurity agency's publication list (ENISA), and a Japanese emergency-response team's English site (JPCERT/CC). No private data, no real company names, no live incident logs — just published guidance.

What the output looks like. Given the starter phrase "The incident response team," the trained AI continues it like this:

"The incident response team should be communicated based on the organization's that may be team shared ability to be detected an a bout of events such as the incident data..."

It reads like English. It uses the right vocabulary ("incident," "organization," "detected," "team"). It sounds professional. But look closely: the words drift ("leaderss," "appliencies"), the sentences don't quite connect, and — most importantly — **none of it refers to anything real**. There is no organization, no incident, no actual procedure. It has learned the *shape* of the language, not the *facts*.

That the text is obviously imperfect is on purpose. We deliberately chose the smallest, crudest version of this technology (letter-by-letter, not word-by-word) so that its mistakes stay visible. A bigger, fancier model would produce smoother, more convincing text — and that would be *more* dangerous, because a tired reader might mistake convincing nonsense for real guidance. The visible mistakes are a safety feature: they make it impossible to forget this is make-believe.

What went wrong along the way (and how it was fixed). Three things broke while building this, all caught before the final results:

1. Early on, the AI produced gibberish studded with control codes like

<BOS> and <EOS> — because the code was accidentally sampling from an *untrained* version of the model. Fixed by sampling from the trained model.

1. Even the trained model would occasionally spit out those control codes,

because the "guess the next letter" step wasn't blocked from picking them. Fixed by forbidding those codes in the guessing step.

1. Some characters came out as garbage symbols (a side effect of extracting

text from PDF documents, which sneaks in invisible font codes). Fixed by stripping those invisible codes out of the training material.

How well it did. On a standard measure for this kind of model (how "surprised" it is by each letter, in units called "bits per character"), it reached **1.53**. Lower is better; this is a solid result for a small model on a small corpus — comparable to well-known reference models on similar tasks.

The one-sentence takeaway. This project shows a small AI can learn the *style* of a security analyst's writing from public documents, and it does so honestly — keeping its mistakes visible so no one confuses its fluent make-believe for real security advice.

(The sections below are written for a technical reader.)

1. Overview

This project implements a character-level Transformer decoder that learns to generate prose in the *response* genre of security operations — the dated, status-driven, paragraph-form language a Security Operations Center (SOC) analyst uses when writing an incident escalation summary. The task is next-character language modeling: given a context of up to 128 characters, predict the distribution over the next character, then sample autoregressively to produce text.

The generative approach is Transformer-based generation, one of the three options on the project rubric (GAN, VAE, or Transformer) [Udacity, 2026]. A Generative Adversarial Network was rejected because text GANs are notoriously unstable to train and prone to mode collapse, which is an unacceptable risk for a capstone deliverable that must converge. A Variational Autoencoder was rejected because text VAEs produce "blurry" outputs — averaging over the latent space yields mid-word mush — a weaker demonstration than a decoder-only model. Character-level (rather than subword/BPE) tokenization was chosen deliberately so that the model's failure modes remain visible in every sample: a subword model at this corpus scale would produce text fluent enough to be mistaken for real advisory prose, which is precisely the failure mode this project's operator-centered framing is designed to expose and avoid.

The end user the system trains for is the SOC analyst who writes escalation summaries at 3 a.m. This project does **not** write that summary. Its job is upstream: demonstrate that a generator can be trained on the operator's genre, so a later retrieval-grounded summarizer can be trusted to sound correct.

2. Dataset Description

The corpus is a public, real, non-synthetic collection of security incident-handling documents, assembled by a catalog-driven downloader (`src/dataset.py` → `SOURCE_CATALOG`).

The sources and their licenses:

Source	Kind	License	Attribution
NIST SP 800-61 Rev. 2 — <i>Computer Security Incident Handling Guide</i>	PDF	Public domain (U.S. Govt)	NIST, 2012
NIST SP 800-61 Rev. 3 — <i>Incident Response Recommendations for CSF 2.0</i>	PDF	Public domain (U.S. Govt)	NIST, 2024
ENISA publications index	HTML	CC BY 4.0	ENISA
JPCERT/CC English home	HTML	CC BY-NC 4.0 (non-commercial)	JPCERT/CC

A live fetch returns all four sources, yielding ****574 records and ~339,000 characters**** (338,715 characters after stripping Private-Use-Area font glyphs that the PDF extractor emits as raw

codepoints). The resulting character vocabulary is **100 tokens**: 96 unique characters plus four special tokens (<PAD>, <BOS>, <EOS>, <UNK>).

Genre choice rationale. NVD/CVE descriptions were considered first and rejected. NVD is *disclosure* prose ("a vulnerability in X allows Y via Z"), which is a different register from the *response* prose the operator writes ("an incident was detected at... the team contained it by..."). Training on the wrong genre would teach the model the wrong surface form. NIST SP 800-61 is the gold-standard public reference for the incident-response craft and is the correct genre. The corpus is ~96% NIST SP 800-61 by character count — a genre bias documented honestly as a limitation in Section 6.

Two planned sources were rejected and documented in the code for transparency: the SANS Internet Storm Center diary archive and the CISA cybersecurity advisories listing are both JavaScript-rendered single-page applications with no static links to follow, so scraping them would require a headless browser, which is out of scope for this project.

3. Model Design and Training Approach

Architecture

The model is a decoder-only GPT-style Transformer with the following design choices, each grounded in established practice:

- **Token + learned positional embedding.** Positional information is added by a

learned embedding rather than the sinusoidal scheme of the original Transformer [Vaswani et al., 2017]. For a small model with a short, fixed context (128 tokens), learned positional embeddings are simpler and perform comparably.

- **Pre-LayerNorm blocks.** Each of the six blocks applies LayerNorm *before*

the attention and feed-forward sublayers, with residual connections around each. Pre-LN is the modern standard because it stabilizes training without requiring a learning-rate warmup [Xiong et al., 2020].

- **Causal multi-head self-attention via scaled_dot_product_attention.** The

causal mask is delegated to PyTorch's fused SDPA kernel with `is_causal=True`, which is both correct and faster than a hand-built boolean mask. The attention mechanism itself follows [Vaswani et al., 2017].

- **Weight-tied LM head.** The output projection reuses the token-embedding

weight matrix (transposed), the GPT-2 trick [Radford et al., 2019]. This saves parameters and ties the input and output views of each character, which empirically improves language-model

quality.

- **Initialization.** Weights are drawn from $N(0, 0.02)$; residual-projection

weights are scaled by $1/\sqrt{2N}$ so the variance of the residual stream stays roughly constant through the stack. This scheme follows nanoGPT [Karpathy, 2022] and removes the need for warmup.

Hyperparameter	Value
Context length (<code>block_size</code>)	128 characters
Model width (<code>d_model</code>)	192
Layers	6
Attention heads	6
Feed-forward width (<code>d_mlp</code>)	768 ($4 \times d_{\text{model}}$)
Dropout	0.1
Unique parameters	~2.68 million

Training

The corpus is sliced into non-overlapping 128-character windows (each window stores `block_size + 1` tokens so the targets are the input shifted by one). This produces 2,625 windows, split 90/10 into training and validation by a seeded `random_split`. The model is trained for 30 epochs with the Adam optimizer ($lr = 1e-3$, no warmup), batch size 64, on a CUDA GPU.

Metric. Training and validation loss are reported in ****bits per character (bpc)****, computed as `loss / ln(2)`. bpc is the standard information-theoretic unit for character language modeling and matches the reporting convention of Karpathy's char-RNN and nanoGPT benchmarks [Karpathy, 2015; Karpathy, 2022], so the number is comparable across the literature. Top-1 next-character "accuracy" is intentionally **not** reported: it is a vanity metric on this task because space and a handful of common letters dominate the vocabulary, inflating the score without reflecting model quality.

A note on the split. Windows are sliced from one continuous corpus, so adjacent windows share text. For a leakage-sensitive task this shared structure could inflate metrics — a lesson documented elsewhere in this capstone from a smoke-detection audit where a random split produced a 0.9998 F1 that collapsed to near-zero sensitivity on held-out independent recording sessions. Here the goal is *genre learning*, not cross-document generalization, and the validation loss is reported only as a convergence signal, not as a generalization claim. The distinction is documented to keep the metric honest.

4. Output Evaluation and Interpretation

Quantitative results

Over 30 epochs the loss falls steadily and the train/validation gap stays small, indicating stable convergence without severe overfitting (dropout 0.1 provides light regularization appropriate to the small corpus):

Epoch	Train bpc	Validation bpc
1	4.53	4.00
10	2.62	2.45
20	1.74	1.72
30	1.39	1.53

A final validation bpc of **1.53** is a solid result for a character language model on a ~339K-character corpus. For comparison, Karpathy's char-RNN on the much larger Shakespeare corpus reaches approximately 1.5 bpc [Karpathy, 2015]; this model is competitive despite a smaller and more specialized corpus, which is consistent with the genre being more regular and repetitive than free literature.

Qualitative evaluation

Samples are generated from the prompt *"The incident response team "* at three temperatures (0.7, 1.0, 1.3) with `top_k = 20`. A representative sample at temperature 0.7:

"The incident response team should be communicated based on the organization's that may be team shared ability to be detected an a bout of events such as the incident data the organization coordination are an incident response team members. The organizational for each sensitive contained with appliencies are leaderss of the re..."

Strengths. The model reliably produces real English words and has clearly absorbed the character-level distribution of the genre. It uses incident-response vocabulary ("incident response team", "organization", "communication", "detected", "contained"), maintains a formal technical register, and occasionally reproduces the dated, structured document shape that NIST publications use. Even at higher temperature the tone stays professional.

Failure cases. The same sample exposes the model's limits, which are the point of the design:

- *Factual nonsense.* The prose reads as fluent incident-handling English but

refers to nothing real — there is no organization, no incident, and the "procedures" are pattern-matched, not retrieved. A character language model learns the surface form of the genre, not the underlying facts [Marcus, 2018].

- *Spelling drift*. Words are frequently close-but-wrong: "leaderss",

"appliencies", "adlidade", "calailidaTable". These are visible artifacts of character-level generation and would be hidden by a subword tokenizer.

- *Drift and repetition*. At longer sample lengths the prose drifts off-topic

or loops on a phrase, a consequence of the 128-character context window being too small to hold a long argument.

- *Rare-character misplacement*. Curly quotes, em-dashes, and the handful of

Unicode marks (§, ®) appear at wrong moments because they are rare in the training data and the model has not learned their correct usage.

These failure modes are **visible in every sample**, and that visibility is the design intent: a subword model would hide them behind fluency, but character-level generation keeps the boundary between the model's pattern-matching and real knowledge legible. This legibility is what makes the model safe to reason about when handing off to a later retrieval-grounded summarizer.

5. Ethical Considerations and Responsible Use

Ethical concern: synthetic text mistaken for real guidance

The most important ethical concern for this system is **misuse through misattribution**: a reader could mistake the model's fluent, genre-correct output for real security guidance or a real incident report. The samples *sound* authoritative — formal, technical, structured — yet they contain no verified information. Any factual claim in a generated sample (a product name, a CVE identifier, a recommended procedure) is coincidence, not knowledge [Marcus, 2018]. If such text were pasted into an analyst's escalation summary without verification, it could propagate fabricated "facts" into a real incident response.

This concern is tied directly to this project's data, model, and outputs. It is the reason the model is character-level rather than subword: a more fluent generator would be *more* dangerous, not less, because its outputs would be harder to distinguish from real advisory prose. The visible spelling drift and factual nonsense in the samples above are a safety property, not a defect.

Responsible-use reasoning

Three design choices address the concern:

1. The model is positioned upstream of the operator-facing text generator.

This project does not feed its own samples into any analyst-facing system. In the later synthesis project, the component that produces analyst-facing text is a *retrieval-grounded* summarizer that cites real policy by document ID — generated text is never the source of truth. The role of this project is methodological: to demonstrate that a generator can learn the operator's genre, and to bound what generated text can and cannot do.

1. Every sample is labeled as synthetic. The notebook's qualitative section

and this report both state explicitly that outputs are modeling artifacts, never authoritative security guidance.

1. Scope minimization in the corpus. The corpus contains no personal data,

no real organization names, and no operational logs — it is published guidance text only. This limits both the privacy surface and the chance that the model memorizes and reproduces real sensitive content. It is the same scope-minimization principle applied to the broader capstone system.

The CC BY-NC 4.0 license on the JPCERT/CC source also imposes a non-commercial-use constraint on any derivative of this corpus, which is respected by limiting the project to academic use and citing the source here and in the code.

6. Limitations and Future Improvements

Limitations

- **Corpus is single-genre and ~96% NIST.** The model has effectively learned

one editorial voice (NIST's). It has not seen the variety of styles a real analyst encounters (vendor advisories, internal postmortems, CSIRT write-ups), so its range is narrow.

- **Small context window.** The 128-character context limits the model's

ability to maintain coherence over a paragraph. Drift and repetition are direct consequences.

- **Character-level generation produces spelling artifacts.** Words like

"leaderss" and "apliencies" are inherent to the tokenization choice. They are a feature for legibility but a limitation for any use that requires clean prose.

- **No quantitative evaluation of coherence.** The project reports bpc (a

likelihood metric) and qualitative samples, but not a structured measure of semantic coherence or factuality, because the latter is not well-defined for synthetic advisory prose and the model is explicitly not fact-grounded.

- **Two planned corpus sources were unreachable.** The SANS ISC diary archive

and the CISA advisories listing are JavaScript-rendered and could not be scraped without a headless browser, so the corpus could not be diversified within scope.

Future improvements

- **Diversify the corpus.** Add SANS handler diaries (the ISC diary archive at

<https://isc.sans.edu/diaryarchive.html> is the best candidate, e.g.

<https://isc.sans.edu/diary/Why+Ask+Credentials+If+There+Are+Secret+Codes/33118/>)

and a redacted internal postmortem corpus, provided a static-render path is found. This would broaden the editorial voice the model learns.

- **Run a subword (BPE) variant as a controlled comparison.** Report its higher

fluency honestly as a *trade-off against legibility of failure modes*, not as a free upgrade — the comparison would quantify exactly what char-level sacrifices and what it gains in safety.

- **Train longer and report the bpc plateau.** The validation curve had not

fully plateaued at 30 epochs; a longer run would establish where overfitting begins and whether dropout should be increased.

- **Add a coherence/factuality guardrail.** Even though this model is upstream

of the analyst-facing generator, a classifier that flags likely-fabricated content would strengthen the responsible-use story.

7. Citations and References

This project's own code, training logs, and generated outputs are not cited.

References

Karpathy, A. (2015). *The Unreasonable Effectiveness of Recurrent Neural Networks* (char-RNN). Retrieved from <https://karpathy.github.io/2015/05/21/rnn-effectiveness/>

Karpathy, A. (2022). *nanoGPT: the simplest, fastest repository for training medium-sized GPTs.* Retrieved from <https://github.com/karpathy/nanoGPT>

Marcus, G. (2018). *Deep Learning: A Critical Appraisal*. arXiv preprint arXiv:1801.00631. Retrieved from <https://arxiv.org/abs/1801.00631>

National Institute of Standards and Technology. (2012). *Security and Privacy Controls for Federal Information Systems and Organizations — SP 800-61 Rev. 2: Computer Security Incident Handling Guide.* U.S. Department of Commerce. Public domain. Retrieved from <https://csrc.nist.gov/pubs/sp/800/61/r2/final>

National Institute of Standards and Technology. (2024). *SP 800-61 Rev. 3: Incident Response Recommendations and Considerations for Cybersecurity Risk Management.* U.S. Department of Commerce. Public domain. Retrieved from <https://csrc.nist.gov/pubs/sp/800/61/r3/final>

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language Models are Unsupervised Multitask Learners* (GPT-2). OpenAI. Retrieved from <https://openai.com/research/better-language-models>

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*. In Advances in Neural Information Processing Systems (NeurIPS 2017). Retrieved from <https://papers.nips.cc/paper/7181-attention-is-all-you-need>

Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L., & Liu, T. (2020). *On Layer Normalization in the Transformer Architecture.* In Proceedings of ICML 2020. Retrieved from <https://arxiv.org/abs/2002.04745>

ENISA (European Union Agency for Cybersecurity). *Publications index*. Licensed CC BY 4.0. Retrieved from <https://www.enisa.europa.eu/publications>

JPCERT Coordination Center. *English-language site*. Licensed CC BY-NC 4.0. Retrieved from <https://www.jpccert.or.jp/english/>